Algorithm 1 PolySAE Training (Shared-U)

Input: activations $\{\boldsymbol{x}\}$, ranks (R_1, R_2, R_3), sparsity K, learning rate η
Initialize $\boldsymbol{U} \leftarrow \mathrm{qr}_+(\boldsymbol{U}_{\mathrm{rand}})$; $\lambda_2 \leftarrow -0.5$; $\lambda_3 \leftarrow 0.5$
for each minibatch $\boldsymbol{x}$ **do**
  // Encode (P1: linear encoder with norm rescaling)
  $\boldsymbol{h} \leftarrow \boldsymbol{E}^\top \boldsymbol{x} + \boldsymbol{b}_{\mathrm{enc}}$
  Compute decoder norms $\boldsymbol{d} \in \mathbb{R}^{d_{\mathrm{sae}}}$: $d_i = \|\mathrm{PolyDec}(\boldsymbol{e}_i)\|_2$
  $\boldsymbol{z} \leftarrow \mathrm{TopK}(\mathrm{ReLU}(\boldsymbol{h} \odot \boldsymbol{d}), K)$
  // Decode (P2–P4: polynomial, shared, hierarchical)
  $\boldsymbol{y}_1 \leftarrow (\boldsymbol{z}\,\boldsymbol{U})\,\boldsymbol{C}^{(1)\top}$
  $\boldsymbol{A}_2 \leftarrow \boldsymbol{z}\,\boldsymbol{U}_{:,1:R_2}$
  $\boldsymbol{y}_2 \leftarrow (\boldsymbol{A}_2 * \boldsymbol{A}_2)\,\boldsymbol{C}^{(2)\top}$
  $\boldsymbol{A}_3 \leftarrow \boldsymbol{z}\,\boldsymbol{U}_{:,1:R_3}$
  $\boldsymbol{y}_3 \leftarrow (\boldsymbol{A}_3 * \boldsymbol{A}_3 * \boldsymbol{A}_3)\,\boldsymbol{C}^{(3)\top}$
  $\boldsymbol{y} \leftarrow \boldsymbol{b}_{\mathrm{dec}} + \boldsymbol{y}_1 + \lambda_2\,\boldsymbol{y}_2 + \lambda_3\,\boldsymbol{y}_3$
  // Update with manifold retraction (P4)
  $\mathcal{L} \leftarrow \|\boldsymbol{y} - \boldsymbol{x}\|_2^2 +$ architecture specific regularizations (e.g. Matryoshka)
  Update all parameters via $\nabla\mathcal{L}$
  $(\boldsymbol{Q}, \boldsymbol{R}) \leftarrow \mathrm{qr}(\boldsymbol{U})$
  $\boldsymbol{S} \leftarrow \mathrm{diag}(\mathrm{sgn}(\mathrm{diag}(\boldsymbol{R})))$
  $\boldsymbol{U} \leftarrow \boldsymbol{Q}\boldsymbol{S}$
end for

A. PolySAE Algorithm

We provide the full training algorithm for PolySAE in Algorithm 1, detailing the encoding, polynomial decoding, and optimization steps used throughout all experiments.

B. Implementation Details

Architecture and Sparsification. We train standard sparse autoencoders (SAEs) and PolySAEs with a latent width of $16{,}384$ and sparsity level $K = 64$. Encoders use one of three sparsification strategies: Top-K (Gao et al., 2025), BatchTopK (Bussmann et al., 2024), or Matryoshka (Bussmann et al., 2025b). For Top-K and BatchTopK, the K largest activations per token (or batch) are retained and the remainder zeroed. All models are trained on residual-stream activations extracted from pretrained language models.

LLMs. We evaluate SAEs and PolySAEs on a standard set of pretrained language models spanning a range of scales: GPT-2 Small (Radford et al., 2019), Pythia-410M and Pythia-1.4B (Biderman et al., 2023), and Gemma-2-2B (Gemma Team, 2024). For each model, we extract residual-stream activations from a single transformer layer chosen near the center of the network, following the methodology of Dunefsky et al. (2024).

PolySAE Decoder Ranks. The rank configurations used in our experiments are:

- GPT-2 Small (Radford et al., 2019): $(768, 64, 64)$
- Pythia-410M (Biderman et al., 2023): $(1024, 128, 128)$
- Pythia-1.4B (Biderman et al., 2023): $(2048, 128, 128)$
- Gemma-2-2B (Gemma Team, 2024): $(2304, 128, 128)$

Training Setup. All models are trained using the Adam optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.999$, a constant learning rate of 3×10^{-4} with no warmup or decay schedules. We use a batch size of 4096 tokens and a context length of 128. We apply gradient clipping with a maximum norm of 1.0 to stabilize training. No weight decay or L1 regularization is applied to the

encoder or decoder weights. Training runs for 5×10^8 tokens for Gemma-2-2B and Pythia models, and 3×10^8 tokens for GPT-2 Small, following the protocol used in the main experiments.

Datasets. For Gemma-2-2B and GPT-2 Small, training data is drawn from OpenWebText (Gokaslan et al., 2019). For Pythia-410M and Pythia-1.4B, we use an uncopyrighted variant of the deduplicated Pile (Gao et al., 2021). Reconstruction is evaluated on held-out data from the same distribution as training.

Evaluation. We evaluate learned representations using SAEBench (Karvonen et al., 2025). Reported metrics include reconstruction error on held-out data and sparse probing performance on six classification tasks: Bias in Bios (De-Arteaga et al., 2019), AG News (Zhang et al., 2015), EuroParl (Koehn, 2005), GitHub programming languages (CodeParrot, 2022), Amazon Sentiment, and Amazon-15 (Hou et al., 2024).

Implementation. Our training pipeline extends `SAELens` (Bloom et al., 2024) to support PolySAE while preserving the standard SAE training interface. PolySAE differs from standard SAEs only in the decoder; all other components, including the encoder, sparsification strategy, optimizer, and evaluation pipeline, are shared across models.

C. Extended Qualitative Analysis

We present an extended qualitative analysis of the interaction structure learned by PolySAE. The analysis proceeds hierarchically, first examining second-order (pairwise) interactions and then extending to third-order (triplet) compositions. Throughout, we compare PolySAE to a vanilla Top-K SAE trained under identical conditions.

C.1. Second-Order Analysis

We begin by analyzing pairwise interactions to assess whether PolySAE captures compositional structure beyond surface-level co-occurrence.

Setup. Both models are applied to 1M documents from OpenWebText. Features are ranked by total activation mass, and the top 10,000 are retained, yielding approximately 5×10^7 candidate feature pairs.

Interaction Strength. For PolySAE, we quantify the strength of a feature pair (i, j) using the learned quadratic decoder weights:

$$B_{ij} = \lambda_2 \left\| (\boldsymbol{u}_i \odot \boldsymbol{u}_j)^\top \boldsymbol{C}^{(2)} \right\|_2, \tag{5}$$

which reflects how much decoder capacity is assigned to that interaction. For the vanilla SAE, which lacks explicit interaction parameters, we use empirical feature covariance,

$$\mathrm{Cov}_{ij} = \mathbb{E}[z_i z_j] - \mathbb{E}[z_i]\mathbb{E}[z_j], \tag{6}$$

as a proxy for pairwise structure.

Relation to Co-occurrence. We independently estimate empirical co-occurrence by counting positions where both features appear in the Top-K active set. For the vanilla SAE, covariance is strongly correlated with co-occurrence ($r = 0.82$), indicating that pairwise structure largely mirrors frequency. In contrast, PolySAE's interaction strengths show negligible correlation with co-occurrence ($r = 0.06$), suggesting that learned interactions reflect structure beyond surface statistics.

Qualitative Regimes. The weak coupling between interaction strength and frequency allows us to identify qualitatively distinct regimes. Of particular interest are *latent* interactions: feature pairs with strong learned interactions despite low empirical co-occurrence. These pairs often correspond to meaningful compositional patterns that are not recoverable from frequency alone.

Examples. For interactions above the 80th percentile in B_{ij}, we extract representative contexts in which both features co-activate. We mark the target token in each sentence and label features by their top-activating tokens. Comparing these contexts with vanilla SAE activations highlights cases where PolySAE captures relationships that the linear model does not.

Table 5. **Compositional Interactions Captured by PolySAE.** PolySAE features (A and B) bind in context to represent specific compositional concepts. Vanilla SAE features (with high-frequency features filtered) fail to capture these compositions.

Poly Feature A	Poly Feature B	Context	Vanilla SAE
[star, stars]	[coffee, tea]	We've all certainly heard of beers brewed with espresso, but how about one with an espresso shot poured over the top? **Starbucks**	[Apple, Google]
[officially, ically]	[traditional, conventional]	It's hard to say what's most impressive about Eduardo Garcia. The class**ically**-trained chef spent years cooking aboard yachts	[newly, ly]
[surgery, repair]	[Trans, LGBT]	Some in the transgender community are worried a suspicious fire at a Montreal clinic will add delays to an already lengthy process to get gender reassignment **surgery**	[birth, baby]
[DNA, genetic]	[mod, mods]	Activists are opening up a new front in their campaign against genetic modification. The latest target is genetically-**mod**ified trees, which scientists believe could bring huge sustainability	[modified, edit]
[secret, hidden]	[Snowden, WikiLeaks]	On May 24th PBS aired a Frontline documentary about alleged Wikileaker Bradley Manning called "**WikiSecrets**"	[secret, secrets]
[business, businesses]	[man, woman]	By Joseph George. The business**man** dad of the boy who drove a Ferrari and was arrested by police in Kerala, India	[man, President]

C.2. Third-Order Analysis

We next examine whether third-order interactions refine or disambiguate pairwise compositions.

Candidate Selection. We focus on latent second-order pairs—those with high interaction strength but low co-occurrence—and identify corpus positions where both features are simultaneously active.

Triplet Scoring. Within these contexts, we evaluate all co-active third features using the learned cubic decoder:

$$\mathrm{Gamma}(f_1, f_2, k) = \lambda_3 \left| (\boldsymbol{u}_{f_1} \odot \boldsymbol{u}_{f_2}) \cdot \boldsymbol{u}_k^\top \boldsymbol{C}^{(3)\top} \right|. \tag{7}$$

For each pair, we retain the third feature with the highest score, after filtering stopword-like features.

Interpretation. The resulting triplets are consistently interpretable, with the third feature modulating the meaning of the pair rather than introducing unrelated content. Common patterns include entity–attribute–domain and subject–object–context structures. Representative examples are shown in Table 10, illustrating how higher-order interactions sharpen and contextualize pairwise compositions.

C.3. Additional Second-Order Interaction Examples

Tables 5–9 show additional second-order interaction examples. Each row highlights a token where two PolySAE features are simultaneously active. Across these tables, the interacting features are typically more specific than the corresponding vanilla SAE features in the same context. The vanilla SAE often activates on a single high-level, morphological, or broadly related feature, while PolySAE activations reflect a more refined decomposition at the highlighted token.

C.4. Additional Third-Order Interaction Examples

Tables 10 and 11 present further third-order examples. Each row shows contexts in which three PolySAE features co-activate at the same token. In these cases, the activated features vary with context and appear more specific than the corresponding vanilla SAE activations, which often capture only one component or default to generic features.